

W7D1 — Python & AI

Setup del laboratorio Python AI e debug dello script Gemini

Sommario

L'esercitazione richiedeva di predisporre un ambiente Python isolato sulla macchina virtuale Kali, configurare l'accesso alle API Gemini di Google e rendere funzionante uno script di chatbot fornito a lezione, che il docente aveva segnalato come non funzionante a causa di cambiamenti nelle API.

L'analisi dello script ha individuato un identificativo di modello non più esistente e l'assenza di gestione degli errori di rete.

Sono stati provati quattro identificativi differenti prima di individuare quello stabile e accessibile, interrogando l'endpoint di elenco dei modelli.

1. Ambiente di lavoro

Le attività sono state svolte su una macchina virtuale Kali Linux ospitata tramite UTM su sistema host macOS. La shell in uso è zsh, versione di Python 3.13. La generazione della chiave API è avvenuta sul sistema host tramite browser, mentre tutte le operazioni di sviluppo ed esecuzione sono state effettuate all'interno della macchina virtuale.

Si segnala che il materiale didattico illustra la condivisione degli appunti tramite il menu di VirtualBox; utilizzando UTM tale menu non è presente e il trasferimento della chiave è stato gestito diversamente.

2. Predisposizione dell'ambiente Python

È stato creato un virtual environment dedicato, in modo da isolare le dipendenze del progetto dall'installazione di sistema, e vi sono state installate le librerie richieste. L'attivazione del virtual environment è verificabile dal prefisso (venv_ai) nel prompt.

```
python -m venv venv_ai
source venv_ai/bin/activate
pip install google-genai pandas numpy
```

```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ nano debug_gemini.py  
(kali@kali)-[~]  
$ python debug_gemini.py  
Traceback (most recent call last):  
  File "/home/kali/debug_gemini.py", line 1, in <module>  
    from google import genai  
ModuleNotFoundError: No module named 'google'
```

Aprendo un nuovo terminale, il virtual environment non risulta più attivo e l'interprete di sistema non trova le librerie installate al suo interno. A differenza della variabile d'ambiente contenente la chiave, resa permanente tramite il file di configurazione della shell, l'attivazione del virtual environment va ripetuta a ogni nuova sessione di terminale.

3. Configurazione della chiave API

La chiave è stata generata da Google AI Studio sul sistema host e impostata come variabile d'ambiente all'interno della macchina virtuale, poi resa permanente aggiungendo la relativa istruzione al file di configurazione della shell.

```
export GEMINI_API_KEY="<chiave>"  
echo export GEMINI_API_KEY="\${GEMINI_API_KEY}" >> ~/.zshrc
```

```
(venv_ai)kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ export GEMINI_API_KEY: [ API KEY OSCURATA ]  
(kali@kali)-[~]  
$ echo "export GEMINI_API_KEY="\${GEMINI_API_KEY}" >> ~/.zshrc  
(kali@kali)-[~]  
$ source venv_ai/bin/activate  
(venv_ai)-(kali@kali)-[~]  
$ nano hello_gemini.py  
(venv_ai)-(kali@kali)-[~]  
$ python hello_gemini.py  
Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use AFC in Chat.send_message. Similarly, direct use of AFC in Models.generate_content_stream is not recommended. Instead, we recommend to use AFC in Chat.send_message_stream.
```

4. Analisi dello script fornito

Lo script proposto a lezione è stato eseguito senza modifiche, per documentare il malfunzionamento segnalato dal docente prima di intervenire.

```
(venv_ai)kali@kali: ~  
Session Actions Edit View Help  
(kali㉿kali)-[~]  
$ source venv_ai/bin/activate  
(venv_ai)-(kali㉿kali)-[~]  
$ cat > chatbot_originale.py << 'EOF'  
from google import genai  
import os  
  
client = genai.Client(api_key=os.environ["GEMINI_API_KEY"])  
  
from google.genai import types  
config = types.GenerateContentConfig(  
    system_instruction="Sei un esperto Cyber Security Analyst."  
)  
  
while True:  
    domanda = input("Tu: ")  
    if domanda.lower() == "esci":  
        print("Arrivederci!")  
        break  
  
    response = client.models.generate_content(  
        model="gemini-3-flash",  
        contents=domanda,  
        config=config  
    )  
    print(f"AI: {response.text}\n")  
EOF  
  
(venv_ai)-(kali㉿kali)-[~]  
$ python chatbot_originale.py  
Tu: che cosa vuol dire poisoning?  
Direct use of automatic function calling (AFC) in Models.generate_content is not re  
commended. Instead, we recommend to use AFC in Chat.send_message. Similarly, direct  
use of AFC in Models.generate_content_stream is not recommended. Instead, we recom  
mend to use AFC in Chat.send_message_stream.  
Traceback (most recent call last):  
File "/home/kali/chatbot_originale.py", line 17, in <module>  
    response = client.models.generate_content(  

```

L'esecuzione si interrompe con un'eccezione. La parte significativa del messaggio si trova in fondo alla traccia di stack:

```
(venv_ai)kali@kali: ~  
Session Actions Edit View Help  
470, in __call__  
    do = self.iter(retry_state=retry_state)  
File "/home/kali/venv_ai/lib/python3.13/site-packages/tenacity/__init__.py", line  
371, in iter  
    result = action(retry_state)  
File "/home/kali/venv_ai/lib/python3.13/site-packages/tenacity/__init__.py", line  
413, in exc_check  
    raise retry_exc.reraise()  
File "/home/kali/venv_ai/lib/python3.13/site-packages/tenacity/__init__.py", line  
184, in reraise  
    raise self.last_attempt.result()  
File "/usr/lib/python3.13/concurrent/futures/_base.py", line 449, in result  
    return self.__get_result()  
File "/usr/lib/python3.13/concurrent/futures/_base.py", line 401, in __get_result  
    raise self._exception  
File "/home/kali/venv_ai/lib/python3.13/site-packages/tenacity/__init__.py", line  
473, in __call__  
    result = fn(*args, **kwargs)  
File "/home/kali/venv_ai/lib/python3.13/site-packages/google/genai/_api_client.py",  
line 1511, in _request_once  
    errors.APIError.raise_for_response(response)  
File "/home/kali/venv_ai/lib/python3.13/site-packages/google/genai/errors.py", li  
ne 173, in raise_for_response  
    cls.raise_error(response.status_code, response.json, response)  
File "/home/kali/venv_ai/lib/python3.13/site-packages/google/genai/errors.py", li  
ne 202, in raise_error  
    raise ClientError(status_code, response.json, response)  
google.genai.errors.ClientError: 404 NOT_FOUND. {'error': {'code': 404, 'message':  
'models/gemini-3-flash is not found for API version v1beta, or is not supported for  
generateContent. Call ModelService.ListModels to see the list of available models  
and their supported methods.', 'status': 'NOT_FOUND'}}  
(venv_ai)-(kali@kali)-[~]  
$
```

Il messaggio comunica che il modello richiesto non è disponibile per la versione dell'API in uso e suggerisce esplicitamente di interrogare il servizio di elenco dei modelli per conoscere quelli effettivamente utilizzabili. Si tratta di un errore di classe `ClientError`, ossia imputabile alla richiesta e non al servizio.

5. Individuazione dei modelli disponibili

Si è seguita l'indicazione contenuta nel messaggio d'errore, interrogando l'API per ottenere l'elenco dei modelli associati alla chiave in uso e filtrando quelli che supportano la generazione di contenuti.

```
(venv_ai)kali@kali: ~  
Session Actions Edit View Help  
ne 202, in raise_error  
    raise ClientError(status_code, response_json, response)  
google.genai.errors.ClientError: 404 NOT_FOUND. {'error': {'co  
'models/gemini-3-flash is not found for API version v1beta, or  
generateContent. Call ModelService.ListModels to see the list  
and their supported methods.', 'status': 'NOT_FOUND'}}  
  
(venv_ai)-(kali@kali)-[~]  
$ python -c "  
from google import genai, os  
" 2>/dev/null; python -c "  
import os  
from google import genai  
c = genai.Client(api_key=os.environ['GEMINI_API_KEY'])  
for m in c.models.list():  
    if 'generateContent' in (m.supported_actions or []):  
        print(m.name)  
" | grep flash  
models/gemini-2.5-flash  
models/gemini-2.5-flash-preview-tts  
models/gemini-flash-latest  
models/gemini-flash-lite-latest  
models/gemini-2.5-flash-lite  
models/gemini-2.5-flash-image  
models/gemini-3-flash-preview  
models/gemini-3.1-flash-lite-preview  
models/gemini-3.1-flash-lite  
models/gemini-3.1-flash-image-preview  
models/gemini-3.1-flash-image  
models/gemini-3.1-flash-lite-image  
models/gemini-3.5-flash  
models/gemini-3.5-flash-lite  
models/gemini-omni-flash-preview  
models/gemini-3.6-flash  
models/gemini-3.7-flash  
models/gemini-3.1-flash-tts-preview  
  
(venv_ai)-(kali@kali)-[~]  
$
```

L'elenco chiarisce l'origine del problema: non esiste alcun modello con l'identificativo indicato nella slide, mentre risulta presente una variante con suffisso di anteprima. È dunque plausibile che il materiale didattico facesse riferimento a un modello in versione preliminare, successivamente rinominato o promosso a versione stabile con una numerazione differente.

6. Selezione del modello e correzione script

L'individuazione dell'identificativo corretto ha richiesto quattro tentativi.

Primo tentativo. L'identificativo indicato nella slide restituisce errore 404: il modello non esiste.

Secondo tentativo. Si adotta l'alias che punta all'ultimo modello Flash stabile. La chiamata funziona, ma nel corso delle prove si manifestano ripetuti errori 503, dovuti al sovraccarico del servizio. Trattandosi dell'alias al modello più recente, e quindi più richiesto, si è ipotizzato che un modello meno recente risultasse meno congestionato.

Terzo tentativo. La scelta di un modello di generazione precedente produce un esito inatteso.

```
(venv_ai)-(kali@kali)-[~]
└─$ python chatbot_gemini.py
Chatbot avviato (modello: gemini-2.5-flash). Scrivi 'esci' per terminare.

Tu: cosa vuol dire poisoning?
[errore nella chiamata alla API]: 404 NOT_FOUND. {'error': {'code': 404, 'message': 'This model models/gemini-2.5-flash is no longer available to new users. Please update your code to use models/gemini-3.6-flash for the latest features and improvements.'}, 'status': 'NOT_FOUND'}}
Tu: 
```

Il modello in questione compariva nell'elenco ottenuto in precedenza pur non essendo utilizzabile: l'elenco riporta dunque i modelli esistenti, non necessariamente quelli accessibili alla chiave: le credenziali di nuova creazione non hanno accesso ai modelli in via di dismissione.

Quarto tentativo. Seguendo l'indicazione ricevuta si adotta il modello suggerito, che risulta stabile e correttamente accessibile.

```
(venv_ai)-(kali@kali)-[~]
└─$ sed -i 's|MODEL = "gemini-2.5-flash"|MODEL = "gemini-3.6-flash"|' chatbot_gemini.py

(venv_ai)-(kali@kali)-[~]
└─$ head -8 chatbot_gemini.py
import os
import sys

from google import genai
from google.genai import types

MODEL = "gemini-3.6-flash"
```

7.Verifica finale

Lo script corretto è stato eseguito ponendo una domanda di ambito cybersecurity.

```
Session Actions Edit View Help
└─$ python chatbot_gemini.py
Chatbot avviato (modello: gemini-3.6-flash). Scrivi 'esci' per terminare.

Tu: cosa vuol dire poisoning?
AI: Nel contesto della **cybersecurity**, il termine **Poisoning** (dall'inglese **avvelenamento**) indica una classe di attacchi, manipolati o malevoli all'interno di un sistema, di una memoria cache o di un database.

L'obiettivo dell'attaccante è "inquinare" la fonte da cui il sistema (o l'utente) attinge le informazioni, costringendolo a siti malevoli o far prendere decisioni sbagliate a un algoritmo.

Per fare un'analogia: è come versare del veleno in un acquedotto cittadino. Chiunque berrà da quell'acquedotto subirà le co
```

Per verificare il mantenimento del contesto è stata formulata una seconda domanda contenente un riferimento implicito alla risposta precedente, senza ripetere l'argomento. La domanda, isolata dal contesto, sarebbe priva di significato.


```

4. **Per le Cache Web:** Corretta configurazione degli header HTTP e disabilitazione del caching per le richieste incoeren
Tu: quale di questi tipi è più pericoloso in una rete aziendale?
AI: In una rete aziendale, la risposta varia in base all'architettura dell'azienda, ma da un punto di vista tattico e di im
ntendersi il titolo di "più pericolosi".
Tuttavia, tra tutti, **l'ARP Poisoning è storicamente il più devastante dall'interno**, mentre il **Data Poisoning (AI)** r
Ecco un'analisi dettagliata del perché, valutata in base a **frequenza, impatto e facilità di esecuzione**:
—

```

Il modello risponde riprendendo puntualmente le quattro categorie che aveva elencato nel turno precedente e costruendo su di esse una classifica di rischio.

```

### Il Verdetto dell'Analista
Se dovessi stilare una classifica di rischio reale per una **tipica rete aziendale oggi**:
1. **ARP Poisoning:** 🟡 **Rischio Immediato Altissimo.** È il metodo primario usato dai ransomware per diffondersi in tutta l'aziend
2. **DNS Poisoning:** 🟡 **Rischio Sistemico Alto.** Meno frequente oggi grazie a DNSSEC e DoH, ma catastrofico se riesce.
3. **Data Poisoning:** 🟡 **Rischio Futuro/Specifico.** Critico oggi solo per aziende fortemente orientate all'AI, ma diventerà mains
4. **Web Cache Poisoning:** 🟢 **Rischio Circoscritto.** Affligge principalmente la reputazione o i clienti che usano i servizi web d
**Come si neutralizza il pericolo maggiore?*
L'adozione di un modello **Zero Trust** (non fidarsi di nulla, nemmeno della rete locale) e la cifratura del traffico interno (TLS/HTT

```

8.Script finale

```

import os
import sys

from google import genai
from google.genai import types

MODEL = "gemini-3.6-flash"

SYSTEM_PROMPT = """
Sei un esperto Cyber Security Analyst.
Analizza i dati forniti e rispondi in modo tecnico ma comprensibile.
Se non sei sicuro, dillo esplicitamente.
"""

def get_client():
    api_key = os.environ.get("GEMINI_API_KEY")
    if not api_key:
        print("Errore: GEMINI_API_KEY non impostata.",
file=sys.stderr)
        sys.exit(1)
    return genai.Client(api_key=api_key)

def main():
    client = get_client()
    config =
types.GenerateContentConfig(system_instruction=SYSTEM_PROMPT)
    chat = client.chats.create(model=MODEL, config=config)

    print(f"Chatbot avviato (modello: {MODEL}). Scrivi 'esci' per
terminare.")

    while True:
        try:
            domanda = input("Tu: ").strip()
        except (EOFError, KeyboardInterrupt):

```

```

        print("\nArrivederci!")
        break

    if not domanda:
        continue

    if domanda.lower() in ("esci", "exit", "quit"):
        print("Arrivederci!")
        break

    try:
        response = chat.send_message(domanda)
    except Exception as err:
        print(f"[Errore nella chiamata alle API]: {err}",
file=sys.stderr)
        continue

    print(f"AI: {response.text or '[nessuna risposta]'}")

if __name__ == "__main__":
    main()

```

11. Conclusioni

L'obiettivo dell'esercitazione è stato raggiunto: l'ambiente Python isolato è operativo, l'accesso alle API Gemini è configurato in modo persistente e lo script del chatbot funziona correttamente, applicando il ruolo definito e mantenendo il contesto conversazionale.